

AIAC _ ASSIGNMENT _1.2

Name: E.Anjani

Batch : 32

HNO : 2303A52419

TASK 1:

#PROMPT AND CODE

```
ASS_1.2 > TASKS.PY > ...
1 # write a python program for factorial of a number without functions
2
3 num = int(input("Enter a number: "))
4 factorial = 1
5
6 for i in range(1, num + 1):
7     factorial *= i
8
9 print("Factorial of", num, "is", factorial)
10
```

#OUTPUT

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/ASS_1.2/TASKS.PY
Enter a number: 5
Factorial of 5 is 120
PS D:\courses\AIAC>
```

#OBSERVATION

- Copilot was helpful for generating logic quickly.
- It provided syntactically correct Python code.
- However, it sometimes used extra variables or repeated logic.
- It does not always follow optimized best practices automatically.
- But for beginners, it simplifies coding and improves productivity.
-

#TASK 2:

#PROMPT

```
write a python program to find the factorial of a number using loops and variable only
donts define any functions optimize this code existing code with simplified logic and
reliable
```

#CODE:

```
# Get the number from user input
n = int(input("Enter a number: "))

# Handle edge cases for reliability
if n < 0:
    print("Factorial is not defined for negative numbers.")
elif n == 0:
    print("The factorial of 0 is 1")
else:
    # Initialize factorial to 1
    factorial = 1
    # Use a loop to calculate factorial (simplified and efficient for small n)
    for i in range(1, n + 1):
        factorial *= i
    # Print the result
    print(f"The factorial of {n} is {factorial}")
```

#OUTPUT

```
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe d:/Course/AIAC/fact.py
Enter a number: 6
The factorial of 6 is 720
(AIAC_env) PS D:\Course\AIAC> █
```

#OBSERVATION

- The optimized version reduces unnecessary variables. Loop logic is clearer and more beginner-friendly.

#TASK 3:

#PROMPT AND CODE

```
1  # Write a factorial program using a function
2
3  def factorial(n):
4      if n == 0:
5          return 1
6      else:
7          return n * factorial(n-1)
8
9  num = int(input("Enter a number: "))
10 print("Factorial of", num, "is", factorial(num))
```

#OUTPUT

```

PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/ASS_1.2/
Enter a number: 7
Factorial of 7 is 5040
PS D:\courses\AIAC>

```

OBSERAVTION

The code is converted into different functions with main block calling and improving readability

#TASK 4 :

TECHNICAL REPORT

The procedural program places all logic in the main flow, making it simple but less organized. It becomes harder to reuse and debug because everything is written in one block. The modular version uses a function, which improves clarity and separates logic from input/output. This makes the code easier to maintain, reuse, and test. Modular design is also better suited for larger projects because updates can be made inside the function without affecting the whole program. Overall, the modular approach is cleaner and more efficient than the procedural version.

#TASK 5:

#PROMPT AND CODE

```

ASS_1.2 > TASK4.py > ...
1  # Generate a python program for factorial of a number using iterative approach
2
3  def factorial(n):
4      result = 1
5      for i in range(1, n + 1):
6          result *= i
7      return result
8
9  num = int(input("Enter a number: "))
10 print("Factorial of", num, "is", factorial(num))
11
12 # Generate a python program for factorial of a number using recursive approach
13
14 def factorial_recursive(n):
15     if n == 0:
16         return 1
17     else:
18         return n * factorial_recursive(n-1)
19
20 num = int(input("Enter a number: "))
21 print("Factorial of", num, "is", factorial_recursive(num))

```

#OUTPUT :

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/ASS_1.2/
Enter a number: 4
Factorial of 4 is 24
Enter a number: 7
Factorial of 7 is 5040
```

#OBSERVATION :

- Iterative version uses a loop and updates product step by step.
- Recursive version calls itself until base condition ($n == 1$).
- Both produce same output but behave differently internally.